SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA15
COURSE OUTCOME:	**CO5:** Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)		
WEIGHTAGE / MARKS:	Weightage: 20% | Total Marks: 30	**TAXONOMY / COUNT:**	Dave's Level 3 (Precision) | Q. Count: 5

CODE OF CONDUCT

I **Aakash R** / **192524462** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Aakash R

1. Uneven Input Splits and Idle Reducers

Likely Design Error:

The input data is highly unbalanced, or the input split/block size is poorly configured. This causes some mappers to receive much more data than others, leading to a data-skew problem. As a result, some reducers finish quickly while others remain overloaded or idle.

Corrective Actions:

- Use appropriate HDFS block/input split sizes.
- Identify and remove data skew.
- Use a suitable custom partitioner to distribute keys evenly across reducers.
- Increase the number of reducers when required.
- Use combiners to reduce intermediate data before shuffling.
- Balance the input files across HDFS.

2. HDFS Data Unavailability After One Node Failure

Probable Design Error:

The HDFS replication factor may be too low, such as replication factor = 1, or replicas may have been placed poorly. If the only copy of a block is on the failed DataNode, that data becomes unavailable.

Another possible issue is poor NameNode design, such as having only a single NameNode without a properly configured Active/Standby High Availability (HA) setup.

Corrective Actions:

- Set an appropriate replication factor, commonly 3 for critical data.
- Ensure replicas are distributed across different DataNodes and racks.
- Use HDFS rack awareness to avoid single-rack failure vulnerabilities.
- Configure NameNode High Availability (HA) with active and standby NameNodes via ZooKeeper / QJM.
- Regularly monitor missing or under-replicated blocks.

3. Duplicate Records in ETL/NiFi Workflow

Likely Causes:

- The NiFi flow may process the same file or message more than once.
- The source system may resend records during retries.
- Incorrect processor relationships or retry configurations may cause duplicate processing.
- No unique identifier or deduplication mechanism is used.
- The analytics database may not enforce uniqueness constraints.

Recommended Fixes:

- Use a unique record ID (UUID/Hash) for every record.
- Configure NiFi processors and routing relationships correctly.
- Use appropriate retry and failure handling mechanisms.
- Maintain checkpoints/state cache to avoid reprocessing ingested files.
- Implement deduplication prior to loading data into the analytics store.
- Add unique constraints, upsert, or merge logic in the target database.
- Design the ETL process to be idempotent, so reprocessing does not duplicate records.

4. YARN Resource Starvation

Likely Scheduling Error:

The cluster may be using an inappropriate scheduler configuration (such as FIFO) or over-allocating resources to individual jobs. Without proper multi-tenant queue management, a single user's workload can monopolize CPU and memory, starving other jobs.

Better Resource Management Approach:

- Configure CapacityScheduler or FairScheduler instead of default FIFO.
- Create separate queues for different users, teams, or departments.
- Set minimum guaranteed and maximum burst resource limits for queues.
- Configure explicit CPU (vCores) and memory allocation bounds.
- Set application-level resource limits and user resource limits.
- Use queue priorities appropriately.
- Monitor cluster utilization actively through YARN ResourceManager.

> **Recommended Approach:** Use **CapacityScheduler** with strict resource quotas and separate queues when predictable multi-tenant resource allocation is required.

5. Backup Restores Outdated Data

Probable Design Flaw:

The backup system may be using stale backups, incorrect replication synchronization, or backups that lack proper versioning/point-in-time snapshots. If replication is asynchronous and latest mutations have not synced before failure, recovery restores an outdated state.

Corrective Actions:

- Schedule backups frequently according to the required Recovery Point Objective (RPO).
- Use incremental backups in conjunction with periodic full backups.
- Maintain multiple versioned point-in-time snapshots.
- Verify that replication is fully synchronized before relying on it for recovery.
- Regularly test backup restoration workflows.
- Maintain comprehensive backup metadata and timestamps.

- Follow a **3-2-1 backup strategy**: 3 copies of data, on 2 different media types, with 1 copy kept off-site.

> **Result:** A properly designed replication and backup architecture guarantees current, recoverable, and verified data integrity during disaster recovery.